

Bridge Day 3 INSTRUCTOR KEY

Raw Input, Type Evidence, Calculations, and F-Strings

Wednesday, July 8, 2026 • 20 points • target 17 minutes

Name		UIN		Section	
------	--	-----	--	---------	--

Assessment boundary and evidence source

Contract: 07a04826c975

Cumulative foundations: print, input, str, float, truthiness, f-strings

Scaffold level: complete validated rate program

Evidence source: the matching v5 workbook readiness/evidence pages and VIP activities 18.3, 18.4, 18.5, 18.6, 18.7. Show intermediate state for trace credit. Program credit requires one coherent solution, a stated assumption when the prompt leaves one implicit, and at least one test whose expected result can distinguish correct from incorrect logic.

Item 1. Trace raw text, conversion, and formatted output. - 4 points

```
raw = "18.5"
value = float(raw)
adjusted = value + 1.5
print(type(raw).__name__, type(value).__name__)
print(f"{adjusted:.1f}")
```

Question: Give the type and value at each named state, then write exact output. **Model response:** raw remains the string 18.5. value is the float 18.5, and adjusted is the float 20.0. Exact output lines are str float and 20.0.

Item 2. Separate truthiness from an approval rule. - 4 points

```
approved_text = "no"
print(bool(approved_text))
print(approved_text == "yes")
```

Question: Give both results and explain why a nonempty input string is not reliable approval evidence. **Model response:** A nonempty string is truthy, so bool(approved_text) is True even when the text is no. The explicit comparison to yes is False. Exact output: True, then False.

Item 3. Evaluate a multi-step Python math expression. - 4 points

```
side_a = 5
side_b = 12
diagonal = (side_a ** 2 + side_b ** 2) ** 0.5
print(f"{diagonal:.2f}")
```

Question: Show the squared terms, the sum, the square-root step, and exact output. **Model response:** The squared terms are 25 and 144, whose sum is 169. Raising 169 to 0.5 gives 13.0. Two-decimal formatting prints 13.00.

Complete program - 8 points

- Read distance and time as floating-point inputs.
- If time is not positive, print invalid and do not calculate speed.
- Otherwise compute $\text{speed} = \text{distance} / \text{time}$.
- Print Speed: followed by the result to exactly two decimal places.

Model program

```
distance = float(input())
time = float(input())

if time <= 0:
    print("invalid")
else:
    speed = distance / time
    print(f"Speed: {speed:.2f}")
```

Reasoning and test evidence

- The time guard precedes division, so zero and negative denominators never reach the calculation.
- For distance 125 and time 10, speed is 12.5 and the f-string prints Speed: 12.50. For time 0, the only output is invalid.

Rubric: 2 input/conversion, 2 protective time guard, 2 correct calculation/branch placement, 2 exact label and precision.